جامعة الامير سلطان
PRINCE SULTAN UNIVERSITY

CS316
INTRODUCTION TO AI AND DATA SCIENCE

CHAPTER 2
Pandas Data Structures

LECTURE 1
Pandas For Data Analytics
Create, Access, Load and Export Data

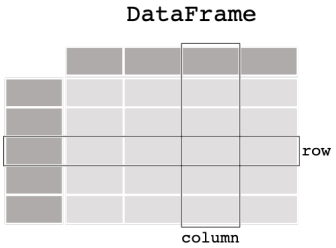
Unorganized data

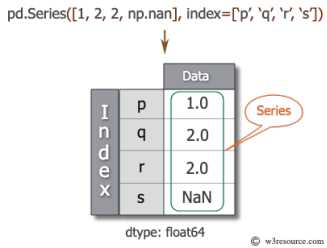
There needs to be 3 cols: index, outcome, design and criteria like age has to be greater than 65

Do not distribute or share any slide without permission of the

What is Pandas?

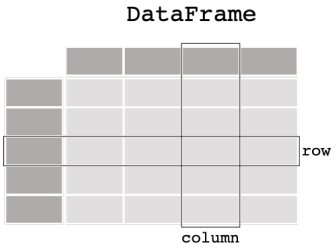
- **Pandas Overview**
  - Python library for **data manipulation** and **analysis**.
  - Built on **NumPy**, leveraging high-performance arrays.
- **Why Pandas?**
  - **Efficient** for working with structured data.
  - Simplifies tasks like data cleaning, exploration, and transformation.
  - Suitable for small to large datasets.

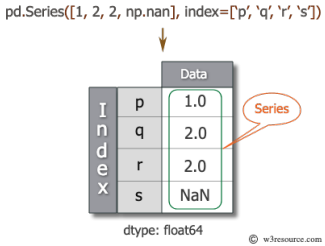




Why Use Pandas?

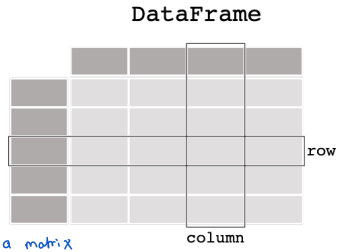
- **Advantages of Pandas**
  - **Simplifies complex tasks** like merging, reshaping, or filtering data.
  - **High flexibility**, handles missing data, time series, etc.
  - Supports **custom data transformations** using lambda functions.
- **Powerful I/O Tools**
  - Easily reads/writes to formats like **CSV**, **Excel**, **SQL**, and **JSON**.
  - Handles **large datasets** efficiently with chunking and memory optimization.



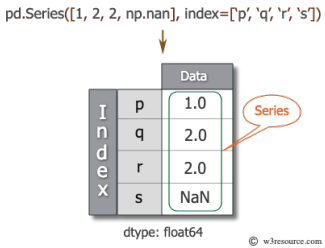


Key Features of Pandas

- **Core Data Structures**
  - **Series**: One-dimensional labeled array. (one column only, could have many rows) like an Array
  - **DataFrame**: Two-dimensional table with labeled axes (rows, columns). like a matrix

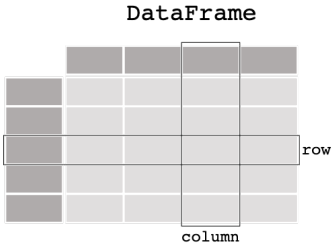


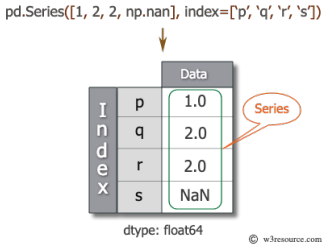
- **Key Functionalities**
  - Handling **missing data** with `.isnull()`, `.dropna()`, `.fillna()`.
  - Flexible **grouping and aggregation** with `.groupby()`, `.agg()`.
  - Efficient **merging and joining** using `merge()` and `join()`.



Comparison with Other Tools

- **Pandas vs Excel**
  - More powerful for **automated, repeatable workflows**.
  - Handles **larger datasets** more efficiently.
- **Pandas vs SQL**
  - SQL for **querying databases**; Pandas for in-memory, **flexible data analysis**.
  - **Both can complement** each other using Pandas' `pd.read_sql()`.
- **Pandas vs Other Python Libraries (e.g., NumPy)**
  - **Pandas** built on **NumPy**, adds easier handling for tabular data.
  - Higher-level API than **NumPy** for **data wrangling tasks**.







Pandas Data Structures Series



SPECIALIZATION

CS316
INTRODUCTION TO AI AND DATA SCIENCE

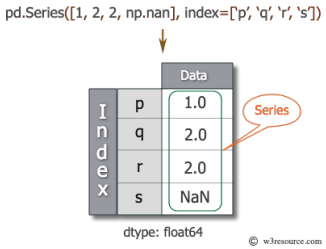
CHAPTER 3
PANDAS AND NUMPY

LECTURE 1

PANDAS

Pandas Series - Definition and Characteristics

- **What is a Series?**
  - **One-dimensional** labeled array, capable of holding any data type.
  - Similar to a **column in a DataFrame** or **an array** in NumPy.
- **Characteristics**
  - Data is aligned to an **index** (row labels).
  - **Heterogeneous data** types: integers, floats, strings, etc.
  - Supports **vectorized operations** (element-wise).



o_of_Bookstore.ipynb ☆

Tools Help

Run all

Google Collab

memorize the code

Along the way you'll practice everything from Chapter 2:

1. Pandas `Series` — creation, access, slicing
2. Pandas `DataFrame` — creation, access, indexing (label- and integer-based)
3. Reading data (CSV, Excel, SQL, JSON)
4. Exporting data (CSV, Excel, SQL, JSON)
5. Handling file paths and URLs

Run the cells in order — later sections build on files created earlier ones.

[3] ✓ 0s

```
import pandas as pd
import numpy as np
import os

# A working folder so all the files we create/read stay organized
# TODO: create a folder named "bookstore_data" to store our files.
# Use os.makedirs(), and make sure re-running this cell doesn't raise an error
# if the folder already exists (hint: there's a parameter for that).
os.makedirs("bookstore_data", exist_ok=True)
print("Pandas version:", pd.__version__)
```

```
Pandas version: 2.2.2
```

A Quick Note :

```
os.makedirs("bookstore_data", exist_ok=True)
```

`exist_ok` is a boolean parameter for `os.makedirs()` that controls what happens if the target directory **already exists**:

- `exist_ok=False` (the default) — raises a `FileExistsError` if the directory already exists.
- `exist_ok=True` — silently does nothing if the directory already exists, instead of raising an error.

It's useful for exactly the situation in the notebook: if you run the cell once, it creates `bookstore_data/`. If you run the notebook again later (or re-run that cell), the folder is already there — without `exist_ok=True`, Python would crash with an error instead of just continuing.

87.35 GB available

ident-_Pandas_Scenario_of_
rive/1vFUJSndKKZEK4VptUR4mTRCPwDYoQ_Kv#scrollTo=50658d06
of_Bookstore.ipynb
ools Help
Run all
Share
RAM
Disk

1.1 Creating a Series — from a list (default integer index)

[4] ✓ 0s

```
# TODO: create a Series from this list of prices
prices = pd.Series([45.0, 60.0, 35.5, 80.0, 25.0])
print(prices)
print("\nType:", type(prices))
```

```
0    45.0
1    60.0
2    35.5
3    80.0
4    25.0
dtype: float64

Type: <class 'pandas.core.series.Series'>
```

Ranked by index

1.2 Creating a Series — from a list, with a custom index

[7] ✓ 0s

```
titles = ["Atomic Habits", "Dune", "The Alchemist", "Sapiens", "1984"]

# TODO: create a Series of the same prices, but use `titles` as the index
book_prices = pd.Series([45.0, 60.0, 35.5, 80.0, 25.0], index=titles, name='price') # Execute the same line without 'name='price'
book_prices
```

make the index as the title
So rank by titles rather than index

	price
Atomic Habits	45.0
Dune	60.0
The Alchemist	35.5
Sapiens	80.0
1984	25.0

dtype: float64

If we remove name='price' we get 0 rather than price

1.3 Creating a Series — from a dictionary

available
10:52 AM Python 3
43°C مشمس ENG 10:53 AM 8/16/2026

Creating a Series

- **From a List**

```python
import pandas as pd
s = pd.Series([1, 2, 3, 4])
```

  - Automatically assigns **default integer index** (0, 1, 2, 3).

- **From a Dictionary**

```python
data = {'a': 10, 'b': 20, 'c': 30}
s = pd.Series(data)
```

  - Index is derived from **dictionary keys**.

Creating a Series

From a List with Custom Index: You can specify a custom index when creating a Series from a list.

python Copy code

```python
import pandas as pd
s = pd.Series([1, 2, 3, 4], index=['a', 'b', 'c', 'd'])
```

From a NumPy Array: You can also create a `Series` from a NumPy array.

python Copy code

```python
import numpy as np
import pandas as pd
data = np.array([1, 2, 3, 4])
s = pd.Series(data)
```

Creating a Series

3. **From a Scalar Value:** A `Series` can also be created from a scalar value, which will be repeated for each index.

```python
import pandas as pd
s = pd.Series(5, index=[0, 1, 2, 3])
```

4. **From Another Series:** You can copy or create a new `Series` from an existing one.

```python
import pandas as pd
original_series = pd.Series([10, 20, 30], index=['x', 'y', 'z'])
new_series = pd.Series(original_series)
```

Accessing Data in a Series

- **Access by Index Label**

```python
s['a']
```

  - Retrieves value associated with index `'a'`.

- **Access by Position**

```python
s[0]
```

  - Retrieves the **first element** in the Series.

- **Slicing**

```python
s[1:3]
```

  - Retrieves a subset of the Series (positions 1 and 2).

Slicing in Pandas Series

- **What is Slicing?**
  - Extracting a subset of elements from a Series based on **index positions** or **labels**.
  - Allows accessing **multiple elements** at once.

- **Slicing by Position (Integer-based indexing)**

```python
s[1:4]
```

  - Returns elements from **index 1 to 3** (excludes end index 4).
  - **0-based indexing**.

Slicing in Pandas Series

- **What is Slicing?**
  - Extracting a subset of elements from a Series based on **index positions** or **labels**.
  - Allows accessing **multiple elements** at once.

- **Slicing by Position (Integer-based indexing)**

```python
s[1:4]
```

  - Returns elements from **index 1 to 3** (excludes end index 4).
  - **0-based indexing**.

Slicing in Pandas Series

- **Slicing by Label (Label-based indexing)**

```python
s['a':'c']
```

  - Returns elements from **'a' to 'c'** (inclusive).

- **Slicing with Steps**

```python
s[0:5:2]
```

  - Retrieves every **2nd element** from index 0 to 4.



Pandas Data Structures
Data Frames



SPECIALIZATION

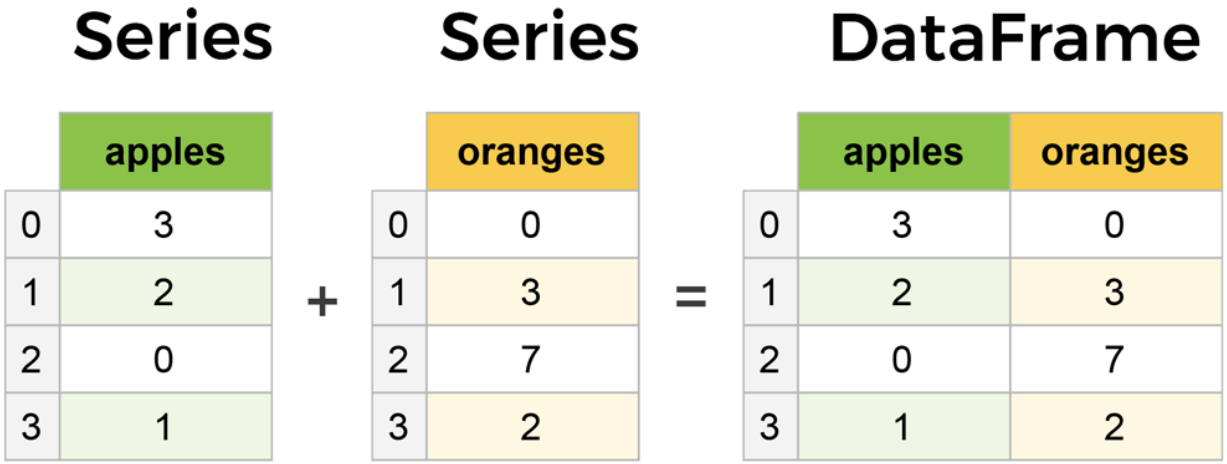
CS316
INTRODUCTION TO AI AND DATA SCIENCE

CHAPTER 3
PANDAS AND NUMPY

LECTURE 1

PANDAS

DataFrame - Definition and Characteristics



DataFrame - Definition and Characteristics

- What is a DataFrame?
  - A **two-dimensional, labeled** data structure in Pandas.
  - **Rows and columns**, similar to a table in a database or an Excel sheet.
  - **Heterogeneous data** types supported across different columns.
- Key Characteristics
  - **Size mutable**: Can add/delete rows or columns.
  - Labeled with **index (rows)** and **columns (headers)**.
  - **Flexible operations** like filtering, grouping, reshaping, and joining.

Creating a DataFrame

- **From a Dictionary**

```python
data = {'Name': ['Alice', 'Bob'], 'Age': [25, 30]}
df = pd.DataFrame(data)
```

  - Dictionary keys become **column labels**, values are **column data**.

- **From a NumPy Array**

```python
import numpy as np
arr = np.array([[1, 2], [3, 4]])
df = pd.DataFrame(arr, columns=['Col1', 'Col2'])
```

  - Requires assigning **column names** explicitly.

- **From CSV or Excel Files**

```python
df = pd.read_csv('file.csv')
df = pd.read_excel('file.xlsx')
```

  - Pandas provides easy methods for importing **external datasets**.

Accessing Data in a DataFrame

- **Accessing Columns**
  - By column label:

```python
df['Name']
```

  - Multiple columns (returns a DataFrame):

```python
df[['Name', 'Age']]
```

- **Accessing Rows**
  - By position (`iloc[]`):

```python
df.iloc[0]  # First row
```

  - By label (`loc[]`):

```python
df.loc['row_label']
```



Pandas Data Structures

Data Frames

Indexing and Selection



SPECIALIZATION

CS316

INTRODUCTION TO AI AND DATA SCIENCE

CHAPTER 3

PANDAS AND NUMPY

LECTURE 1

PANDAS

Label-based Indexing

- **Definition**:
  - `loc[]` selects data by **label** or **conditional statements**.
- **Accessing Rows by Label**:

```python
df.loc['row_label']
```

  - Retrieves a row using a **specific label** from the index.
- **Selecting Specific Columns by Label**:

```python
df.loc[:, 'column_label']
```

  - Retrieves an entire **column** by label.
- **Conditional Selection**:

```python
df.loc[df['Age'] > 25]
```

  - Retrieves rows where the condition is **True**.

Label-based Indexing

Subsetting with `.loc[]`

- Subset specific rows and columns using `.loc[]` (label-based selection).

```python
df.loc[:, ['column1', 'column2']]
```

- Example:

```python
df.loc[:, ['Name', 'Salary']]  # Select all rows for 'Name' and 'Salary'
```

Label-based Indexing

Filtering by Multiple Conditions

- Combine conditions using logical operators:
  - `&` for AND
  - `|` for OR

```python
df[(df['Age'] > 30) & (df['Gender'] == 'Male')]
```

- Example:

```python
df[(df['Age'] > 30) & (df['Salary'] > 50000)]  # Rows where 'Age' > 30 AND 'Sa
```

Integer-based Indexing

- **Definition**:
  - `iloc[]` selects data based on **integer position** (like NumPy arrays).
- **Accessing Rows by Position**:

```python
df.iloc[0]
```

  - Retrieves the **first row** (0-based index).
- **Selecting a Range of Rows and Columns**:

```python
df.iloc[0:3, 1:3]
```

  - Selects rows **0 to 2** and columns **1 to 2**.



Pandas Data Structures
Data Importing in Pandas



SPECIALIZATION

CS316
INTRODUCTION TO AI AND DATA SCIENCE

CHAPTER 3
PANDAS AND NUMPY

LECTURE 1

PANDAS

Reading Data into Pandas - From CSV

1. **From CSV Files (`pd.read_csv()`)**
   - **Description**:
     - CSV (Comma-Separated Values) is one of the most common data formats.
     - `pd.read_csv()` is highly flexible, capable of handling large datasets efficiently.
   - **Basic Usage**:

```python
df = pd.read_csv('data.csv')
```

Reading Data into Pandas - From CSV

- `sep` : Specifies delimiter (default is a comma). For tab-separated values:

```python
df = pd.read_csv('data.tsv', sep='\t')
```

- `header` : Defines which row is treated as the header:

```python
df = pd.read_csv('data.csv', header=0)  # First row is the header
```

- `index_col` : Set a specific column as the DataFrame index:

```python
df = pd.read_csv('data.csv', index_col='ID')
```

- `usecols` : Read only specific columns:

```python
df = pd.read_csv('data.csv', usecols=['Name', 'Age'])
```

Common Parameters

Reading Data into Pandas – From Excel Files

- **Description**:
  - Excel is widely used for data storage and analysis. Pandas allows reading `.xlsx` and `.xls` formats.
  - Requires the **openpyxl** or **xlrd** library for handling Excel files.
- **Basic Usage**:

```python
df = pd.read_excel('data.xlsx')
```

Reading Data into Pandas – From Excel Files

- `sheet_name`: Specifies the sheet to read (can be sheet name or index):

```python
df = pd.read_excel('data.xlsx', sheet_name='Sheet1')
```

  - Use `sheet_name=None` to read **all sheets** into a dictionary.

- `skiprows`: Skip specific rows at the beginning of the file:

```python
df = pd.read_excel('data.xlsx', skiprows=2)  # Skip the first two rows
```

- `usecols`: Select specific columns to load:

```python
df = pd.read_excel('data.xlsx', usecols='A:C')  # Load columns A to C
```

Common Parameters

Reading Data into Pandas – From SQL Databases

- **Description**:
  - SQL databases store structured data, and Pandas can directly query relational databases such as SQLite, MySQL, PostgreSQL, and more.
  - Pandas requires a **database connection** to execute SQL queries.
- **Basic Usage (SQLite Example)**:

python Copy code

```python
import sqlite3
conn = sqlite3.connect('database.db')
df = pd.read_sql('SELECT * FROM table_name', conn)
```

Reading Data into Pandas – From SQL Databases

- **For PostgreSQL/MySQL:**
  - Use libraries like `psycopg2` (PostgreSQL) or `mysql-connector-python` (MySQL):

```python
import psycopg2
conn = psycopg2.connect("dbname=test user=postgres password=secret")
df = pd.read_sql('SELECT * FROM table_name', conn)
```

- **Parameter `params`:**
  - Use with parameterized SQL queries:

```python
df = pd.read_sql('SELECT * FROM table WHERE Age > ?', conn, params=[30]
```

Reading Data into Pandas – From JSON Files

- **Description**:
  - JSON (JavaScript Object Notation) is a lightweight format often used for web APIs.
  - Pandas can read JSON into a DataFrame, converting nested structures into rows and columns.
- **Basic Usage**:

```python
df = pd.read_json('data.json')
```

Reading Data into Pandas – From JSON Files

- `orient` : Specifies the expected structure of the JSON file. Common options:
  - `records` : List of dictionaries (default format).
  - `index` : Data formatted as a dictionary of dictionaries, with row labels.

```python
df = pd.read_json('data.json', orient='index')
```

- `lines` : Set to `True` if JSON is in newline-delimited format:

```python
df = pd.read_json('data.json', lines=True)
```

Reading Data into Pandas – From JSON Files

`orient` Parameter

The `orient` parameter specifies the expected structure (or orientation) of the JSON file. The most common options include:

1. `records` **(default option)**:
   - The JSON file is expected to be a **list of dictionaries**, where each dictionary represents a row in the `DataFrame`.
   - **Structure Example**:

```json
[
    {"column1": value1, "column2": value2},
    {"column1": value3, "column2": value4}
]
```

   - **Code Example**:

```python
df = pd.read_json('data.json', orient='records')
```

Reading Data into Pandas – From JSON Files

2. `index`:

   - The JSON file is expected to be a **dictionary of dictionaries**, where the keys of the outer dictionary represent the row labels (index), and the inner dictionaries contain the column data for each row.
   - **Structure Example**:

     ```json
     {
         "row1": {"column1": value1, "column2": value2},
         "row2": {"column1": value3, "column2": value4}
     }
     ```

   - **Code Example**:

     ```python
     df = pd.read_json('data.json', orient='index')
     ```

Reading Data into Pandas – From JSON Files

`lines` Parameter

The `lines=True` parameter is used when the JSON file is in a **newline-delimited format**. In this format, each line in the file represents a separate JSON object.

- **Structure Example**:

```json
{"column1": value1, "column2": value2}
{"column1": value3, "column2": value4}
```

- **Code Example**:

```python
df = pd.read_json('data.json', lines=True)
```



Pandas Data Structures

Data Exporting in Pandas



SPECIALIZATION

CS316
INTRODUCTION TO AI AND DATA SCIENCE

CHAPTER 3
PANDAS AND NUMPY

LECTURE 1

PANDAS

Data Exporting in Pandas - Exporting to CSV

1. Exporting to CSV

- **Basic Usage:**
  - Save a DataFrame to a CSV file.

python Copy code

```python
df.to_csv('output.csv', index=False)
```

Data Exporting in Pandas - Exporting to CSV

- `sep` : Specify a different delimiter (e.g., tab-separated values `\t`).

```python
df.to_csv('output.tsv', sep='\t', index=False)
```

- `header` : Whether to include the column headers in the output file.

```python
df.to_csv('output.csv', header=False)  # Excludes column names
```

- `index` : Include or exclude the row index in the file.

```python
df.to_csv('output.csv', index=True)  # Includes row indices
```

- `columns` : Select specific columns to save in the CSV.

```python
df.to_csv('output.csv', columns=['Name', 'Age'], index=False)
```

Data Exporting in Pandas - Exporting to CSV

Advanced Parameters:

- `line_terminator`: Customize the line ending (e.g., `\n`, `\r\n`).

```python
df.to_csv('output.csv', line_terminator='\n')
```

- `encoding`: Specify character encoding (useful for non-ASCII data).

```python
df.to_csv('output.csv', encoding='utf-8')
```

Data Exporting in Pandas - Exporting to Excel

2. Exporting to Excel

- Basic Usage:
  - Save a DataFrame to an Excel file.

python Copy code

```python
df.to_excel('output.xlsx', sheet_name='Sheet1', index=False)
```

Data Exporting in Pandas – Exporting to Excel

- `sheet_name`: Set the name of the Excel sheet where data will be written.

```python
df.to_excel('output.xlsx', sheet_name='Data', index=False)
```

- `index`: Include or exclude the row index in the Excel file.

```python
df.to_excel('output.xlsx', index=True)  # Includes row indices
```

Data Exporting in Pandas - Exporting to Excel

- **Advanced Parameters**:
  - `engine`: Specify the engine for writing Excel files (usually **openpyxl** for `.xlsx`). `engine` = the underlying library that Pandas delegates the Excel file-writing task to.

```python
df.to_excel('output.xlsx', engine='openpyxl')
```

  - `startrow` **and** `startcol`: Define the starting row and column for writing data.

```python
df.to_excel('output.xlsx', startrow=2, startcol=1)
```

    - This is useful when you want to write data to a specific location in the sheet.
  - `freeze_panes`: Freeze the top or side rows/columns.

```python
df.to_excel('output.xlsx', freeze_panes=(1, 0))  # Freeze the top row
```

  - `columns`: Select specific columns to save.

```python
df.to_excel('output.xlsx', columns=['Name', 'Salary'])
```

Data Exporting in Pandas - Exporting to SQL

- **Basic Usage**:
  - Save a DataFrame to an SQL table.

```python
df.to_sql('table_name', conn, if_exists='replace', index=False)
```

Data Exporting in Pandas - Exporting to SQL

- `conn` : Database connection object (e.g., from SQLite, MySQL, PostgreSQL).

```python
import sqlite3
conn = sqlite3.connect('database.db')
df.to_sql('employees', conn, index=False)
```

- `if_exists` : What to do if the table already exists.
  - `'replace'` : Drop the table, then recreate and insert data.
  - `'append'` : Add the DataFrame to the existing table.
  - `'fail'` : Raise an error if the table exists.

```python
df.to_sql('employees', conn, if_exists='append', index=False)
```

Data Exporting in Pandas - Exporting to SQL

Advanced Parameters:

- `dtype`: Specify the SQL data type for each column.

```python
df.to_sql('employees', conn, dtype={'Name': String, 'Age': Integer})
```

  - Useful when you want to control how columns are stored in the database.

- `chunksize`: Write the DataFrame in batches to handle large datasets.

```python
df.to_sql('employees', conn, chunksize=500)
```

  - Writes 500 rows at a time, useful for exporting large DataFrames.

- `method`: Specify the SQL insert method.
  - You can optimize bulk inserts using specific database features.

```python
df.to_sql('employees', conn, method='multi')
```

Data Exporting in Pandas - Exporting to SQL

there are three important arguments:

- `'employees'` → the name of the SQL table
- `conn` → the database connection
- `method='multi'` → **how the rows should be inserted**

What does `method='multi'` mean?

Normally, Pandas can insert rows **one at a time**.

For example, imagine your DataFrame contains:

```
Name     Age
Alice    25
Bob      30
John     28
```

With the default method, conceptually, it might perform:

SQL

```sql
INSERT INTO employees VALUES ('Alice', 25);
INSERT INTO employees VALUES ('Bob', 30);
INSERT INTO employees VALUES ('John', 28);
```

With:

Python

```python
method='multi'
```

Pandas tries to send **multiple rows in a single INSERT statement**:

Data Exporting in Pandas - Exporting to JSON

Basic Usage

- Save a DataFrame to a JSON file.

python Copy code

```python
df.to_json('output.json')
```

Data Exporting in Pandas – Exporting to JSON

- `orient`: Controls the structure (format) of the output JSON. Key formats:
  - `'columns'` (default): Columns as keys, with nested row values.

```python
df.to_json('output.json', orient='columns')
```

Output:

```json
{
  "column1": {"0": "value1", "1": "value2"},
  "column2": {"0": "value3", "1": "value4"}
}
```

Data Exporting in Pandas – Exporting to JSON

- `'records'`: List of dictionaries (each row is a dictionary).

```python
df.to_json('output.json', orient='records')
```

Output:

```json
[
  {"column1": "value1", "column2": "value3"},
  {"column1": "value2", "column2": "value4"}
]
```

Data Exporting in Pandas – Exporting to JSON

- `'index'`: Row indices are the keys, with rows as nested dictionaries.

```python
df.to_json('output.json', orient='index')
```

Output:

```json
{
  "0": {"column1": "value1", "column2": "value3"},
  "1": {"column1": "value2", "column2": "value4"}
}
```

Data Exporting in Pandas – Exporting to JSON

- `lines`: Writes the JSON data in a **newline-delimited** format (useful for large datasets and streaming).

```python
df.to_json('output.json', lines=True, orient='records')
```

Output:

```json
{"column1": "value1", "column2": "value3"}
{"column1": "value2", "column2": "value4"}
```

  - Each row is written as a separate JSON object, making it easier to process with streaming applications or when reading line-by-line.

Data Exporting in Pandas - Exporting to JSON

- `date_format`: Control the format of datetime objects in the JSON output.
  - Use `'iso'` for ISO 8601 format or `'epoch'` for UNIX timestamps.

```python
df.to_json('output.json', date_format='iso')
```

  - Outputs dates as `"YYYY-MM-DDTHH:MM:SS"`.
- `double_precision`: Control the precision of floating-point numbers.

```python
df.to_json('output.json', double_precision=2)
```

  - Rounds floating-point numbers to 2 decimal places.
- `force_ascii`: Force encoding to ASCII (useful for non-ASCII characters like emojis).

```python
df.to_json('output.json', force_ascii=False)
```

  - Allows Unicode characters to appear as-is instead of ASCII-encoded characters.

Common Use Cases

1. **Exporting Data to Share**:
    - **CSV**: Best for sharing raw data in plain text format (widely used across applications).
    - **Excel**: Best for reports where formatting, multiple sheets, and formulas may be needed.
    - **SQL**: Best for saving structured data into a relational database for further querying.
2. **Performance Tips**:
    - For **large datasets**, export in **chunks** using `chunksize` to avoid memory issues.
    - When exporting to **Excel**, consider limiting the file size and using CSV for very large datasets.



Pandas Data Structures
Handling File Paths and URLs



SPECIALIZATION

CS316
INTRODUCTION TO AI AND DATA SCIENCE

CHAPTER 3
PANDAS AND NUMPY

LECTURE 1

PANDAS

Handling File Paths

- **Local File Paths**:
  - When working with local files, simply provide the **relative** or **absolute path** to the file.

```python
df = pd.read_csv('data/myfile.csv')  # Relative path
df = pd.read_csv('/home/user/data/myfile.csv')  # Absolute path
```

- **Working with Different Operating Systems**:
  - Use Python's `os` library for **cross-platform compatibility**.

```python
import os
path = os.path.join('data', 'myfile.csv')  # Automatically handles slashes
df = pd.read_csv(path)
```

- **Using `pathlib` for File Handling**:
  - `pathlib` is a modern and intuitive way to handle paths.

```python
from pathlib import Path
path = Path('data/myfile.csv')
df = pd.read_csv(path)
```

Handling URLs

- **Reading Data from a URL**:
  - Pandas can directly read files hosted on the web, such as CSV or Excel files, using a URL.

```python
df = pd.read_csv('https://example.com/data.csv')
```

- **Supported File Types for URLs**:
  - **CSV**: `pd.read_csv()`
  - **Excel**: `pd.read_excel()`
  - **JSON**: `pd.read_json()`
  - Example:

```python
df = pd.read_json('https://example.com/data.json')
```

Handling URLs

- **Handling Authentication for URLs:**
  - For URLs that require authentication (e.g., APIs), use `requests` to handle the download and pass it to Pandas.

python Copy code

```python
import requests
response = requests.get('https://example.com/data.csv', auth=('username', 'pass
with open('localfile.csv', 'wb') as f:
    f.write(response.content)
df = pd.read_csv('localfile.csv')
```

Advanced URL Handling

- **Reading Data from APIs**:
  - Combine Pandas with **web APIs** to read data directly from the web.

```python
import requests
response = requests.get('https://api.example.com/data')
data = response.json()
df = pd.DataFrame(data)
```

- **Reading from Cloud Storage**:
  - Pandas supports reading from cloud storage systems (AWS S3, Google Cloud Storage, etc.) using specialized libraries (e.g., **s3fs** for AWS).

```python
df = pd.read_csv('s3://bucket_name/data.csv', storage_options={'key': 'your_key
```